

CSE 493S Empirical Methods Assignment

Progress Report: Finished Deliverables

Aadyant Maity and Rahul Bonthu

May 1, 2026

1 Part 0: implementation

Part 0 is basically the codebase the rest of the assignment builds on. Everything lives under `deliverables/part_0/`: the usual scripts (`train.py`, `model.py`, `inference.py`, `tokenizer_utils.py`, `arith_data.py`) plus `part_0_1_contract.py` for the autograder-facing API.

We sanity-check with the quick configs from the README, for example:

```
python train.py --config configs/sanity_quick.yaml
python train.py --config configs/sanity_suffix_quick.yaml
```

(run from the `EmpiricalHW/` root, or point paths appropriately). Nothing fancy to report here besides “it trains and the contract loads checkpoints”—the interesting experiments start in Part 1.

2 Section 1.1: modular arithmetic datasets

For §1.1 we generated text files of equations $a \text{ op } b = c$ with arithmetic mod p . Addition and subtraction iterate all pairs (a, b) with $0 \leq a, b < p$. Division skips $b = 0$ and uses $b \in \{1, \dots, p-1\}$, so division sets are a bit smaller than add/sub for the same p .

We shuffled once with seed 0, then split each file **80% / 10% / 10%** train, validation, and test in that order. Every `meta.json` matches the lines in the matching `.txt` files.

2.1 Split sizes

These counts match what is on disk (see `deliverables/part_1/1.1/README.md`).

Table 1: Number of lines per split for each operator and modulus.

p	op	train	val	test
97	+ / -	7527	940	942
97	/	7449	931	932
113	+ / -	10215	1276	1278
113	/	10124	1265	1267

To regenerate from the course workflow:

```
python arith_data.py --seed 0
```

3 Section 1.3: division mod 97 (grokking-style run)

§1.3 trains on division modulo 97. The deliverable bundle is under `deliverables/part_1/1.3/checkpoints/part1_3_div_p97/`, with final weights `ckpt_300000.pt`, `tokenizer.json`, `metrics.csv`, and resolved config metadata.

3.1 Training curves

Figure 1 shows loss and accuracy vs. step from `metrics.csv`. We generated the PNG with `plot_metrics.py` as described in the folder README.

3.2 Numbers at the last logged step

At step 300000 (last row of `metrics.csv`), the CSV reports roughly:

- Train loss ≈ 0.331 , train token acc ≈ 0.888 .
- Val loss ≈ 2.38 , val token acc ≈ 0.560 .
- Test loss ≈ 2.31 , test token acc ≈ 0.566 .
- Train equation acc ≈ 0.618 ; val/test equation acc stayed low on the last rows (this is consistent with the grokking story where memorization on train can run ahead of generalization on full equations).

We are not over-interpreting these here—the figure is the main takeaway for us until we write up discussion elsewhere.

3.3 How to run inference

From `deliverables/part_1/1.3/code/`, the README suggests either free-form generation or `predict_answer` on integers. Example:

```
python inference.py --checkpoint_dir ../checkpoints/part1_3_div_p97 \
  --prompt "3 / 5 = "
```

4 What is still open

- **§1.2:** We have smoke plots and multi-restart curves under `part_1/1.2/plots/`, and intermediate checkpoints under `part_1/1.2/checkpoints/`, but we still need to drop in the true final `ckpt_100000.pt` when the long run finishes.
- **§1.4:** Ablation configs exist; we still need to train, save final `ckpt_300000.pt` for each setting, and copy the bundles into this deliverables tree.
- **Part 2 / Part 3:** Notebook work and the written essay are tracked separately from this summary.

Closing

If someone only reads one thing: Part 0, §1.1, and §1.3 are in good shape in `deliverables/`; the rest is either partial or lives outside this folder. We will update this `.tex` file when §1.2 and §1.4 catch up.

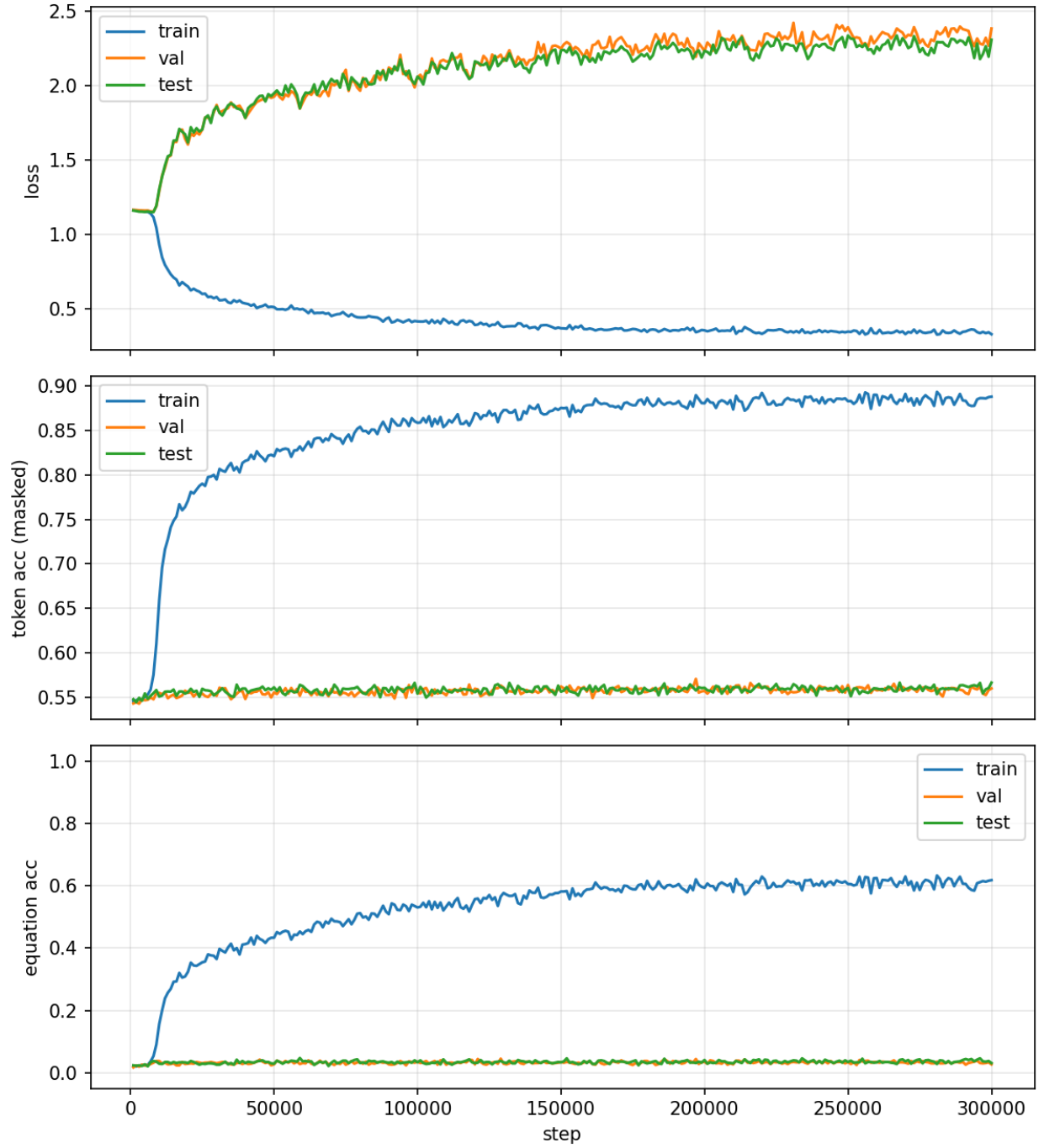


Figure 1: Training / validation / test curves for the division mod 97 experiment (part1_3_div_p97).